One rule, two languages: C++/Kokkos and Rust ports of WAVEWATCH III that must give the Fortran's bits

Matheus Hoffmann

Escola Politécnica, Universidade Federal do Rio de Janeiro, Brazil · ORCID 0009-0009-1056-7661 · Preprint, not peer reviewed · 2026-11-11

Abstract

We report two agent-written ports of the WAVEWATCH III (WW3) spectral wave model that share one acceptance rule: the port must produce the same bits as the Fortran, and a tolerance is a diagnostic, never the verdict. *ww3-gpu* keeps WW3 and lets its Fortran call C++ written with Kokkos, behind one run-time flag per routine; the reference is the Fortran path of the same executable. *chaceon* replaces WW3 with Rust, compiled for the CPU, for SPIR-V (rust-gpu) and for PTX (Rust-CUDA), with no Fortran in its process; the reference is a separately compiled WW3. On the CPU, ww3-gpu's discrete interaction approximation (DIA) of the four-wave transfer, 48.4 % of single-thread self time on a 9 520-point case, is byte-identical on all 100 serial and 25 OpenMP entries of WW3's regression matrix that select it. chaceon runs `ww3_grid`, `ww3_strt`, `ww3_prep` and `ww3_shel` without Fortran and writes 96 output files byte-identical to the reference. Neither has run the whole model on a real GPU, and neither claims a speed-up here. We compare how each project defines and checks its evidence.

1. Why bit for bit

WW3 integrates the action balance equation $\partial N/\partial t + \nabla\cdot(\dot{c}N) = S/\sigma$, where the source term S is the sum of wind input, dissipation, non-linear transfer and bottom terms. A port that changes the order of a floating-point sum changes N, and a forecast system that has been validated against observations would have to be validated again. Both projects therefore refuse to change the Fortran's arithmetic: no reordered reduction, no fused multiply-add (`-ffp-contract=off` on both sides), no widened tolerance. A difference is a defect of the port until two runs of the Fortran itself are shown to differ in the same file.

2. Two shapes for the same rule

	ww3-gpu (C++/Kokkos)	**chaceon (Rust)**
Shape	Inside WW3: the Fortran calls C++ through a `bind(C)` shim, one flag per routine	Replaces WW3: no Fortran in a chaceon process
Reference	NOAA-EMC/WW3 `03adda91` via a fork that adds 5 687 lines and removes none	metocean/WW3 7.14, `92e2cf66`
CPU parity against	the Fortran path of the same executable	a separately compiled reference
Device targets	Kokkos 5.2.2, CUDA backend (`sm_89`); OpenMP on the CPU	rust-gpu (SPIR-V, Vulkan) and Rust-CUDA (PTX)
Spectrum resident on device	yes, on the OpenMP backend	yes, in SPIR-V on CPU Vulkan (lavapipe)
Size of the port	62 257 C++ lines, 355 files (2026-10-14)	29 229 Rust port lines, 5 936 tool lines
Formal methods	SMT queries, exhaustive sweeps	one Lean 4 proof about `f32` division
Licence	MIT, LGPL-3.0 kernels	MPL-2.0

Table 1. Sources: chaceon `docs/WW3_GPU_PARITY_202611.md` (2026-11-09) and README at `0c098fe`; ww3-gpu `kokkos/PORT_STATUS.md` at `58167d7`.

The Kokkos route keeps every WW3 program and swaps routines one at a time, so a single regtest can isolate one routine's effect. The Rust route has to rebuild the programs, the file formats and the time loop before a whole run can be compared, but once it can, there is no Fortran left to share state with the port. ww3-gpu's CPU results for its whole runtime, the resident spectrum included, and their limits are kept in its NOAA progress record [3]; its DIA port is also published on its own, with a comparison against an earlier CUDA kernel of the same routine, as ww3-dia-gpu-v2 [4].

3. Evidence ladders

Each project names its steps, and a step in one ledger is not the same evidence as a step in the other.

Step	**ww3-gpu**	**chaceon**
Code exists	*translated*: 316 routines, 26.2 % of Fortran lines	*translated*: Rust exists, not fully reached
Called by a run	*ported*: 72 routines, 99.3 % of profiled run time	*routine*: every captured call bit-identical on the host and in SPIR-V
Whole runs identical, CPU	*regtested*: 14 routines, 48.6 % of run time	*host*: whole runs `cmp`-identical; 10 080 lines
Identical through a GPU API	none	*spirv*: same files from SPIR-V on CPU Vulkan; 5 189 lines
Identical on a real GPU	*gpu*: none	*cuda*: none

Table 2. ww3-gpu counts from `scripts/port_score.py --write` (profile: `bench_small`, 120×80, 12 h, one thread, gprof, 2026-10-14); chaceon counts from `just port-summary`.

4. What each project found

ww3-gpu: the reference is the optimised code. Reaching byte identity for the DIA required one fact that reading the source could not give: gfortran expands `x**11` as a different chain of multiplications at `-O0` and at `-O3`, and the two chains disagree on 643 427 of 1 000 000 single-precision inputs. Before this was found, the research log explained the residual differences as round-off amplified by WW3; that explanation was wrong. The port's reference is therefore the code WW3's own build produces, read as GIMPLE or disassembly, not the Fortran text. The port has no unit tests of its own: it is judged only by WW3's regression tests, compared by `test.comp`.

chaceon: a run with no Fortran in it. Rust's `ww3_grid`, `ww3_strt`, `ww3_prep` and `ww3_shel`, run in sequence from the namelists, write `mod_def.ww3`, the forcing, gridded and point outputs, the track file and both restarts byte-identical to the reference on 96 files; boundaries give 214 identical comparisons over 34 cases, and the track post-processors 74. `ww3_shel --threads N` and an MPI build give the same bytes. With `W3SRCE`, `W3XYP3` and `W3KTP3` compiled to SPIR-V and run on a CPU Vulkan driver, the files are still identical. Where enumeration cannot reach (2^{64} pairs of `f32` operands), a Lean proof states the division fact the port relies on; it says nothing about whether Rust equals Fortran, which has no usable formal semantics.

Why ww3-gpu has no proof yet, and what comes next. In this first phase ww3-gpu adopted no proof assistant [5]. No proof assistant reads Fortran or C++, so a proof is about a hand-written model and says nothing about the gfortran and nvcc binaries unless the model is shown to match them, which is the open question; bit equality is a property of two binaries built with given flags. For straight-line IEEE-754 code it is decidable: SMT queries in `QF_FP` and sweeps over all 2^{32} `float32` inputs answer it in seconds to minutes, against months per routine for an interactive proof that agents must also write and a person review. A first formal layer sits beside the ports: the Lean 4 `ocean-waves` package [6] proves, with no `sorry` or `axiom`, that $\sigma^2(k)$ is strictly increasing, that $c/2 \le c_g \le c$, that the depth CFL bounds every Courant number by 1, that upwind propagation and a DIA-shaped exchange conserve action, and that the semi-implicit source step stays positive; it runs in exact `Rat` on a reduced model, so it says nothing about bits either. Agda is the next step: it has no 32-bit float, so its role is a total, executable reference semantics of each source term, with table indices typed as `Fin` so that every read is in bounds for every grid. The bits stay with the sweeps, the solver queries and WW3's regression tests.

5. What is not claimed

1. Neither project has run the whole model on a real GPU. chaceon's PTX has never executed; ww3-gpu has run one DIA benchmark kernel on an RTX 4090 [4].
2. No speed-up is claimed. ww3-gpu's recorded timings [3] are marked "regtest pending" by its own rules, and chaceon has no timings.
3. On a GPU, bit identity also needs the device's `expf` and `logf` to return glibc's bits. ww3-gpu routes its kernels to device copies of glibc 2.44's functions, equal to the host's on every input checked, but no device run has compared them yet; until one does, a GPU run is judged by a fixed tolerance.
4. The two references are different WW3 lines (NOAA-EMC and MetOcean), so the two ports are compared by routine name, not by source text: 85 routines are in both ledgers.

6. Use of AI

The C++/Kokkos and Rust code, the shims, the comparison scripts, the investigations and drafts of this text were produced by Claude coding agents (Anthropic), under the author's direction, in the two repositories. The agents' rules, skills and hooks are published in each repository's `AGENTS.md` and `.claude/` directory. Every number above was read from a file of the repository at the commit named; none was run for this note. WW3's physics is used unmodified; WAVEWATCH III is a registered trademark of NOAA.

Availability and references

1. ww3-gpu, github.com/h0ffmann/ww3-gpu, DOI 10.5281/zenodo.23221351.
2. chaceon, github.com/h0ffmann/chaceon at `0c098fe`.
3. A bit-identical Kokkos runtime for WAVEWATCH III: CPU results and five speed-ups. ww3-gpu `docs/NOAA_PROGRESS_RECORD_202610.md` at `58167d7`.
4. ww3-dia-gpu-v2: the whole W3SNL1, inside WAVEWATCH III, bit for bit. github.com/h0ffmann/ww3-dia-gpu-v2 at `32aa2ef`.
5. ADR-0001: No proof language for the GPU port. ww3-gpu `docs/ADRs/ADR-0001-proof-language.md` at `58167d7`.
6. Ocean waves: the physics of a spectral wave model in Lean 4. github.com/h0ffmann/proof, `ocean-waves/` at `b863c15`.